INSTALLATION MANUAL

SQL Query Notebook

Organize, Search & Manage Your SQL Queries

Install the application on your own Google account. Twelve steps, no coding, no server, no cost. You paste two files, run one function, and press Deploy.

Steps	12, plus an optional GitHub step
Time needed	About 25 minutes
Files to paste	2 (`Code.gs` and `Index.html`)
Cost	None — runs inside a normal Google account
You need	A Google account. A work (Workspace) account is strongly recommended
Version	1.0.0

✓ **Nothing here is destructive.** If you get something wrong you can delete the spreadsheet and start again. No other data in your account is touched.

Before you start

Everything you need is in the folder **SQL-Query-Notebook-2-FILES**. There are three files, and only the first two get pasted into Apps Script.

FILE	WHAT IT IS	SIZE
`Code.gs`	The entire backend. All 16 modules merged into one file.	5,649 lines
`Index.html`	The entire front end. All the styling and all 21 screens.	5,072 lines
`appsscript.json`	The manifest. Apps Script creates this for you — you only replace its contents.	544 bytes

How this manual is laid out. Each step gives you the exact actions to take, then a **What you should see** box so you can confirm it worked before moving on. Where something is easy to get wrong, there is a warning box explaining what goes wrong and why.

PHASE	STEPS	WHAT HAPPENS
A · Prepare	1–2	Create the spreadsheet and open the code editor
B · Install the code	3–5	Paste the two files and set the permissions manifest
C · Initialise	6–8	Build the database and load demo data
D · Publish	9–10	Deploy the web app and open it
E · Go live	11–12	Add your team and switch on maintenance

PHASE A · PREPARE

STEP 01 Create the Google Sheet

≈ 1 minute · sheets.google.com

This spreadsheet becomes your database. The application creates all 14 tabs inside it later — you only create the empty file.

1. Go to **sheets.google.com** and click **Blank spreadsheet**.
2. Click the title *Untitled spreadsheet* at the top left.
3. Type this name exactly, then press Enter:

```
SQL Query Notebook DB
```

✓ **What you should see:** An empty spreadsheet named *SQL Query Notebook DB*, with one tab called `Sheet1`.

! Do **not** create any tabs yourself. Step 6 builds all 14 with the correct column headers. Leave `Sheet1` alone — setup deletes it.

STEP 02 Open the Apps Script editor

≈ 1 minute · from inside the spreadsheet

Apps Script is Google's built-in code editor. It is free and already part of your account — there is nothing to install.

1. In the spreadsheet menu bar, click **Extensions → Apps Script**.
2. A new browser tab opens with a file called `Code.gs` containing an empty `myFunction()`.
3. Click *Untitled project* at the top left and rename it:

```
SQL Query Notebook
```

You will use four things in this editor:

- The **+** button next to *Files* — adds a file (step 4)
- The **function dropdown** in the toolbar — picks which function to run (steps 6, 8, 12)
- The **▶ Run** button — runs it
- The **Deploy** button, top right — publishes the web app (step 9)

On the far left is a narrow icon bar. Two icons matter: the **gear** (Project Settings, step 5) and the **clock** (Triggers, step 12).

PHASE B · INSTALL THE CODE

STEP 03 Paste the backend into Code.gs

≈ 3 minutes · 5,649 lines

The editor already has a file called `Code.gs`. You are replacing its contents — do **not** create a new file.

1. Click anywhere inside the code area.
2. Select everything: **Ctrl + A** (Mac: ⌘ + **A**).
3. Press **Delete**. The file should now be completely empty.
4. Open `Code.gs` from the installation folder in Notepad, VS Code, or any text editor.
5. Select all, copy.
6. Click back into the empty Apps Script editor and paste.
7. Wait for the paste to finish, then press **Ctrl + S** to save.

✓ **What you should see:** The file starts with `/**` and the words *SQL QUERY NOTEBOOK — COMPLETE BACKEND*, followed by a contents list. No red error marks in the margin.

! It is a large file. Give the paste a few seconds to finish **before** you save — saving mid-paste stores a truncated file.

If the paste looks cut off. Scroll to the very bottom. The last line should be a closing brace of the `updateSettings` function. If it stops mid-word, select all, delete, and paste again — waiting for the editor to settle before saving.

Finding your way around this file later. It opens with a table of contents, and each of the 16 original modules keeps its own banner. Press **Ctrl + F** and search for a section number to jump straight there.

SEARCH FOR	TAKES YOU TO
`SECTION 01`	Config — schema, permissions, defaults
`SECTION 02`	Utils — spreadsheet access, cache, settings
`SECTION 04`	Auth — identity, sessions, permission checks
`SECTION 05`	Queries — create, search, filter, dashboard

SEARCH FOR	TAKES YOU TO
SECTION 09	Users — accounts, roles, permission matrix
SECTION 15	Setup — `setupSystem()`, `seedDemoData()`
SECTION 16	Code — `doGet()` and all 57 API endpoints

STEP 04 Add Index.html for the front end

≈ 3 minutes · 5,072 lines

This one is a new file, so this time you use the **+** button.

1. Next to **Files** on the left, click **+**, then choose **HTML**.
2. The editor asks for a name. Type exactly this — with **no** `.html`, the editor adds it:

```
Index
```

3. Press Enter. The editor creates `Index.html` with some placeholder HTML.
4. Select all of that placeholder (**Ctrl + A**) and delete it.
5. Open `Index.html` from the installation folder, select all, copy.
6. Paste into the editor and save with **Ctrl + S**.

✖ **The name must be exactly Index, with a capital I.** The application looks the file up by that name. `index`, `Index.html` typed in full, or `MyIndex` all produce a blank page when you deploy — and the error message will not tell you why.

✓ **What you should see:** The Files panel now lists exactly two files — `Code.gs` and `Index.html`. That is the whole application.

STEP 05 Set the manifest

≈ 2 minutes · appsscript.json

The manifest tells Google which permissions the application needs. It is hidden by default, so you switch it on first.

1. Click **Project Settings** — the gear icon in the far-left icon bar.
2. Tick **“Show ‘appsscript.json’ manifest file in editor”**.
3. Click **Editor** (the <> icon) to go back. There is now a third file, `appsscript.json`.
4. Open it, select all, delete.
5. Paste in the contents of `appsscript.json` from the installation folder.
6. Find the `"timeZone"` line. If `Asia/Karachi` is not your timezone, change it.

7. Save with **Ctrl + S**.

> ✓ **What you should see:** The file contains an `"oauthScopes"` list with six entries. Save produces no error.

What each permission is for. You will be asked to approve these in step 7.

PERMISSION	NEEDED BECAUSE
Spreadsheets	The database lives in Google Sheets
Drive	Creates the folder tree, `.sql` files, attachments and backups
Send email as you	Share and update notifications
See your email address	Identifies who is signed in — this is the basis of every permission check
Manage scripts	Installs the hourly session-cleanup job
Connect to an external service	Fetches the `.xlsx` copy of your sheet during a backup

PHASE C · INITIALISE

STEP 06 Run setupSystem()

≈ 1 minute to start · builds the database

This one function creates all 14 tabs with their headers, the four roles, the permission matrix, the starter categories, the Drive folders, and your own administrator account.

1. In the toolbar above the code, open the function dropdown. It currently says `doGet` or `myFunction`.
2. Choose **setupSystem** from the list.
3. Click ▶ **Run**.

> **!** It will stop and ask for permission. That is expected — continue to **step 7**, then come back and press Run again.

If setupSystem is not in the dropdown, the file was not saved or the paste was incomplete. Press **Ctrl + S**, wait for "Saved", then reload the browser tab. If it is still missing, redo step 3.

STEP 07 Authorise the permissions

≈ 2 minutes · once only

Google shows a scary-looking warning here. It appears for every private script that has not gone through Google's public add-on review. It is your own code, in your own account.

1. Click **Review permissions**.
2. Choose your Google account.
3. You will see *"Google hasn't verified this app"*. Click **Advanced** at the bottom left.
4. Click **Go to SQL Query Notebook (unsafe)**.
5. Read the permission list, then click **Allow**.

The **Advanced** link is easy to miss — it is small text at the bottom left of the warning box. Clicking it reveals the "Go to … (unsafe)" link that lets you continue.

Now click ▶ **Run** again. Watch the **Execution log** panel at the bottom.

```
========================================
 SQL Query Notebook — setup complete
```

```
========================================
Spreadsheet : SQL Query Notebook DB
Sheets created  : Queries, Categories, Databases, Users, ...
Seeded          : Roles, Permissions, Categories, Databases,
                  Settings, Admin user (you@company.com)
Drive root      : https://drive.google.com/...
```

✓ **What you should see:** Switch to the spreadsheet tab — it now has 14 tabs with bold headers. A Drive folder called *SQL Query Notebook* now exists with four subfolders.

Is it safe to run setupSystem again? Yes, at any time. It only adds what is missing and never overwrites data. If you ever rename or delete a tab by accident, running it again rebuilds it.

STEP 08 Load the demo data

≈ 1 minute · optional but recommended

Ten example queries, so the application has something to show the first time you open it. Skip this if you would rather start empty.

1. In the function dropdown, choose **seedDemoData**.
2. Click ▶ **Run**.

✓ **What you should see:** The log reports *Inserted 10 demo queries (SQL-000001 … SQL-000010)*, and the `Queries` tab now has ten rows.

! The demo SQL is illustrative. It assumes a courier and logistics schema that will not match your warehouse. Every demo query is tagged `demo` so you can filter and delete them later.

PHASE D · PUBLISH

STEP 09 Deploy as a web app

≈ 3 minutes · the most important settings in this manual

1. Top right, click **Deploy → New deployment**.
2. Click the gear icon next to *Select type* and choose **Web app**.
3. Fill in the three fields exactly as below.

FIELD	SET IT TO
Description	v1.0.0 — any text, for your own reference
Execute as	**Me (your@email.com)**
Who has access	**Anyone within your organisation**

4. Click **Deploy** and authorise again if asked.
5. **Copy the Web app URL.** It looks like `https://script.google.com/a/macros/.../exec`

✖ **"Execute as" must stay "Me".** That setting is what lets the script read the spreadsheet while your users have no access to it at all — which is the entire basis of the permission system. If you set it to *User accessing the web app*, every user would need edit access to the spreadsheet, handing them the whole dataset and letting them rewrite the permission table.

Choosing "Who has access"

OPTION	USE WHEN	EFFECT
Anyone within your organisation	A company Workspace account	Recommended. Google sign-in identifies people reliably
Anyone with a Google account	External collaborators	Google identity may come back empty; the app falls back to application passwords
Only myself	Testing	Nobody else can open it

! Share the **web app URL** with your team. Never share the spreadsheet — anyone with

edit access to it bypasses every permission in the application.

STEP 10 Open it and sign in

≈ 1 minute · first look

1. Paste the Web app URL into a browser tab.
2. Sign in with the same Google account if prompted.

✓ **What you should see:** The dashboard, with your name and an **Admin** badge in the top right, and — if you ran step 8 — ten queries and a set of statistics.

Try these three things to confirm everything works:

- Press **Ctrl + K** and type `rider` — search should find matches inside the SQL.
- Open `SQL-000001`, then **History → Compare** — you should see a coloured line-by-line diff.
- Open the same URL on your phone — the layout should switch to cards with a bottom navigation bar.

Troubleshooting

IF YOU SEE	WHAT IT MEANS AND HOW TO FIX IT
“Your Google account is not registered”	The application working correctly — it only lets registered people in. The email Google reported does not match the `Users` tab. Check the spelling in row 2. If the wrong account ran setup, re-run `setupSystem()` from the correct one.
A blank page	Almost always the HTML file name. It must be exactly `Index` — capital I, no extension typed. Rename it, save, then **Deploy → Manage deployments → edit → New version → Deploy**.
“The application is not set up yet”	`setupSystem()` has not run successfully. Go back to step 6 and check the execution log for an error.
Sign-in loops back to login	The deployment is not “Execute as: Me”. Redeploy with the settings in step 9.

PHASE E · GO LIVE

STEP 11 Add your team

≈ 3 minutes · inside the application

People cannot sign in until you add their email address, even if they are in your organisation.

1. In the sidebar, click **Users**.
2. Click **+ Add User**.
3. Enter their name, work email address and role. Leave the password checkbox unticked — they sign in with Google.
4. Repeat for each person, then send them the web app URL.

ROLE	CAN DO	GIVE IT TO
Admin	Everything, including users, roles, settings and backups	You, and one backup person
Manager	Create, edit and delete any query; restore; export	Team leads
Editor	Create and edit their own queries; share	Analysts who write SQL
Viewer	View, search and copy only	Everyone else

> **!** Always have a **second Admin**. The application refuses to remove the last active administrator, so if you are the only one and you lose access, recovery means editing the spreadsheet by hand.

Test that permissions actually work. Ask a Viewer to open the app. They should see no **New Query** button, no **Edit**, and no *Manage* section in the sidebar. The hidden buttons are only a courtesy — the server refuses the actions too. You can confirm that: open **Activity Logs** and look for `ACCESS_DENIED` entries.

STEP 12 Turn on maintenance

≈ 3 minutes · back in Apps Script

Three one-off jobs. Run each from the function dropdown, exactly as in step 6.

FUNCTION	WHAT IT DOES
`installTriggers`	Adds the hourly job that expires idle sessions

FUNCTION	WHAT IT DOES
`sendTestNotification`	Sends you a test email — confirms notifications work
`showConfiguration`	Prints the configuration to the log so you can check it

Add a weekly backup

1. In the far-left icon bar, click the **clock icon** (Triggers).
2. Click **+ Add Trigger** at the bottom right.
3. Function to run: `runScheduledBackup`
4. Event source: **Time-driven → Week timer →** Sunday, 3am–4am.
5. Click **Save**.

> ✓ **What you should see:** A test email in your inbox, and the Triggers page listing two triggers — `cleanupExpiredSessions` and `runScheduledBackup`.

OPTIONAL · BACK IT UP ON GITHUB

STEP 13 Put it on GitHub

≈ 5 minutes · optional · no command line needed

Apps Script keeps no history you can browse and no way to see what changed between two versions. GitHub gives you both, plus somewhere the code survives if the Google account is ever lost. This step is optional — the application runs perfectly without it.

> ✓ **These files are safe to publish.** No spreadsheet IDs, no Drive folder IDs, no script ID, no passwords, no API keys and no company data. All real IDs live in Script Properties inside your Apps Script project, which never leaves Google.

Option A · Upload in the browser (recommended)

No git install, no command line. Drag and drop.

1. Go to **github.com/new**. Sign up first if you do not have an account — it is free.
2. Repository name:

```
sql-query-notebook
```

3. Description (optional): *SQL query manager on Google Sheets, Apps Script and Drive*
4. Choose **Private** or **Public** — see the table below.
5. Leave *Add a README*, *.gitignore* and *license* all **unticked**. Your folder already has them, and ticking these creates a conflict.
6. Click **Create repository**.
7. On the next page click **uploading an existing file**.
8. Open `E:\SQL Query Notebook\SQL-Query-Notebook-2-FILES` in File Explorer.
9. Select everything *except* the `_to_delete` folder, and drag it onto the GitHub page.
10. Wait for all files to finish uploading, then click **Commit changes**.

> ✓ **What you should see:** The repository page listing `Code.gs`, `Index.html`, `appsscript.json`, `README.md`, `SETUP.md`, `LICENSE`, `preview.html` and a `docs` folder — with your README rendered underneath.

Option B · Command line

If you have Git for Windows installed. Open the folder in File Explorer, type `cmd` in the address bar and press Enter, then run:

```
git init -b main
git add .
git commit -m "SQL Query Notebook v1.0.0"
git remote add origin https://github.com/YOUR-USERNAME/sql-query-notebook.git
git push -u origin main
```

Replace **YOUR-USERNAME** with your GitHub username. GitHub will ask you to sign in the first time you push.

> **!** Delete the `_to_delete` folder first, in File Explorer. It holds a half-built git folder that would confuse `git init`.

Private or public?

CHOOSE	WHEN
Private	The default choice. Only you and people you invite can see it. Everything still works — history, diffs, releases.
Public	Anyone can read it. Fine for this code, which contains no secrets. Choose it only if you actively want to share the project.

> **✖ Never commit .clasp.json.** That file holds your Apps Script project ID. The included `.gitignore` already excludes it, along with `.env`, key files and credential JSON — but check before every upload that you are not adding one by hand.

Keeping it up to date

GitHub does not talk to Apps Script automatically. When you change the code, update both:

1. Edit `Code.gs` or `Index.html` in the Apps Script editor.
2. Copy the changed file back over the one in your local folder.
3. On GitHub, open that file, click the **pencil** icon, paste the new contents, and **Commit changes** with a short note saying what changed.

Because GitHub keeps every version, you can open any file's **History** to see exactly what changed and when — the thing Apps Script cannot show you.

Advanced: clasp. Google’s command line tool removes the copy-paste step entirely. Install with `npm install -g @google/clasp`, sign in with `clasp login`, then create `.clasp.json` holding your Script ID (Project Settings in the editor). After that `clasp push` uploads and `clasp pull` downloads. Worth it if you edit often; overkill if you paste twice a year.

Before you tell people it’s live

Six checks. The first one is the one that matters most.

- ☐ **The spreadsheet is shared with nobody.** Open it, click **Share**, and confirm you are the only person listed. Anyone with edit access here bypasses every permission in the application.
- ☐ **The deployment says** Execute as: Me**.**
- ☐ **Two-factor authentication is on for the Google account you deployed from.**
- ☐ **A Viewer account really cannot see New Query, Edit, or the Manage section.**
- ☐ **There is a second Admin.**
- ☐ **Demo data has been removed, or you have decided to keep it.**

Updating it later

When you change the code, paste the new file in, save, then publish a new version:

```
Deploy → Manage deployments → edit → New version → Deploy
```

> **!** Use **New version** on the deployment you already have. Creating a *New deployment* gives you a different URL and leaves everybody on the old code — a confusing failure to diagnose.

The URL stays the same, your data is untouched, and users get the update on their next page load. To roll back, open the same dialog and pick an earlier version.